Programming with Agencia

Here is how program using Agencia. Agencia is a platform where you create AI "Agents" to interpret user requests and help them with tasks.

Instead of writing complex code, you write "Agents" in a simple checklist format (YAML) and give them instructions using **Parley**, a language designed to be as close to English as possible.

1. What is an Agent?

Think of an Agent as a specific worker with a job description. You give the Agent:

1. **A Name**: What do we call this worker? (e.g., `greet` , `scheduler`)
2. **A Description**: What does this worker do? This helps the AI figure out when to use this agent.
3. **A Job**: What logic does this agent follow? You choose one of two keywords:
    - `template` : Returns the text exactly as you wrote it (after filling in blanks). Use this for fixed responses or logic.
    - `prompt` : Sends your text to an AI. The AI reads it as instructions and generates a smart response.

Here is a simple example:

```yaml
agents:
  # Template: Deterministic output for simple tasks
  greet:
    description: "Greet the user"
    template: |
      Hello! I am an AI assistant. How can I help you today?

  # Prompt: AI-driven output for complex tasks
  storyteller:
    description: "Tell a story"
    prompt: |
      Write a very short story about {{ INPUT }}.
```

When `greet` runs, it always says the exact same thing. When `storyteller` runs, the AI reads your prompt and invents a new story based on the input.

2. Using Parley

Parley is the language we use inside the `template` to make our agents smart and dynamic. Parley instructions are always written inside double curly braces: `{{ ... }}` .

Inputs: Listening to the User

The simplest thing an agent can do is repeat what the user said. We use `INPUT` for this.

```
agents:
  echo:
    description: "Repeat what the user says"
    template: |
      You said: {{ INPUT }}
```

If the user says "I love pizza", the agent replies: "You said: I love pizza".

Structured Inputs: extracting details

Sometimes you want specific details, not just the whole sentence. You can define `inputs` for your agent, and the AI will extract them for you.

```
agents:
  greeter:
    description: "Greet a person by name"
    inputs:
      name:
        description: "The name of the user"
    template: |
      Hello, {{ INPUT name }}! It is nice to meet you.
```

If the user says "My name is Sarah", the AI finds "Sarah" and puts it in `name`. The agent replies: "Hello, Sarah! It is nice to meet you."

Calling Other Agents: Asking for Help

Agents can work together. One agent can `SEND` a message to another agent to get a result.

```
agents:
  time_keeper:
    description: "Get the current time"
    template: |
      It is 12:00 PM.

  greeter:
    description: "Greet the user with the time"
    template: |
      Hello! {{ SEND time_keeper MESSAGE INPUT }}
```

This would output: "Hello! It is 12:00 PM."

You can also save the result of a call to use it later, which makes your instructions easier to read.

```
    template: |
      {{ LET time BE SEND time_keeper MESSAGE INPUT }}
      Hello! I checked the clock and {{ USE time }}.
```

3. Facts: Outputs and Memory

Just like the AI extracts `Inputs` from what the user says, it can also extract `Facts` from what an Agent says (returns). Facts are the "Output" of the agent that we want to save.

To allow an agent to save information for later, you define it in a `facts` section.

```
agents:
  assistant:
    facts:
      username:
        description: "The user's name"
    template: |
      Hello, {{ FACT username }}.
```

If the `username` fact was previously saved as "John", the agent says "Hello, John."

Using Facts

Once a Fact is saved, you can use it in any other agent. You can often leave out the word `FACT` if it's clear what you mean, like inside a list or a check.

To get a fact from a specific agent, use `IN`:

```
    template: |
      I remember your name is {{ FACT username IN assistant }}.
```

4. Making Decisions (Conditionals)

Sometimes you want your agent to say different things based on the situation. We use `IF`, `THEN`, and `ELSE` for this.

```
template: |
  {{ IF INPUT name IS EMPTY THEN }}
    I don't know your name yet.
  {{ ELSE }}
    Hello, {{ INPUT name }}!
  {{ END }}
```

You can check if things are `EMPTY`, `IS` a specific value, or `IS NOT` a value.

```
template: |
  {{ IF FACT status IS active THEN }}
    System is online.
  {{ ELSE }}
    System is offline.
  {{ END }}
```

5. Working with Lists (Iteration)

Often you have a list of items, like a shopping list or a set of reminders. Parley makes it easy to show these lists.

Imagine you have a fact called `shopping_list` with items like "Milk", "Eggs", and "Bread".

By default, lists are shown as bullet points. To show a list, we use the `AS BULLETS` command:

```
template: |
  Here is your list:
  {{ LIST shopping_list AS BULLETS }}
```

Output:

```
Here is your list:
- Milk
- Eggs
- Bread
```

You can also ask for other formats, like sentences:

```
template: |
  {{ LIST shopping_list AS SENTENCES }}
```

Output: `Milk. Eggs. Bread.`

Lists can be declared statically with a block structure.

```
    template: |
      {{ SEND agent LIST}}
        - Milk
        - Eggs
        - Bread
      {{END}}
```

6. Putting it All Together

Here is a more complete example of an agent that takes a coffee order.

```
agents:
  barista:
    description: "Take a coffee order"
    inputs:
      drink:
        description: "Type of coffee (e.g. Latte, Espresso)"
      size:
        description: "Size of the drink"
    template: |
      {{ IF INPUT drink IS EMPTY THEN }}
         What would you like to order?
      {{ ELSE }}
         Okay, keeping it simple. One {{ INPUT drink }} coming up!
         {{ IF INPUT size IS NOT EMPTY THEN }}
            Making it a {{ INPUT size }}.
         {{ END }}
      {{ END }}
```

How it Works Explained

Let's walk through three different examples of how this agent behaves.

Scenario 1: User says "Hello"

- **AI Interpretation**: The AI looks for a drink name but finds none. So `drink` is EMPTY.
- **Logic**:
  - The agent checks: `{{ IF INPUT drink IS EMPTY ... }}`.
  - This is **True**, so it runs the first block of instructions.
- **Agent Says**: "What would you like to order?"

Scenario 2: User says "One coffee please"

- **AI Interpretation**: The AI extracts "coffee" as the `drink`, but finds no size.
- **Logic**:

    - The agent checks: `{{ IF INPUT drink IS EMPTY ... }}`.
    - This is **False**. It skips to the `ELSE` block.
    - It prints: "Okay, keeping it simple. One coffee coming up!"
    - Then it checks the next instruction: `{{ IF INPUT size IS NOT EMPTY ... }}`.
    - This is **False** (size is empty), so it ignores the line inside.
- **Agent Says**: "Okay, keeping it simple. One coffee coming up!"

Scenario 3: User says "I want a large Mocha"

- **AI Interpretation**: The AI extracts "Mocha" as the `drink` and "large" as the `size`.
- **Logic**:
    - The agent checks: `{{ IF INPUT drink IS EMPTY ... }}`. This is **False**.
    - It prints: "Okay, keeping it simple. One Mocha coming up!"
    - Then it checks: `{{ IF INPUT size IS NOT EMPTY ... }}`.
    - This is **True**. It runs the instruction inside.
    - It adds: "Making it a large."
- **Agent Says**: "Okay, keeping it simple. One Mocha coming up! Making it a large."

Appendix A: Parley Commands Reference

INPUT

Get the message from the user or a specific extracted field.

```
# Get the full user input
{{ INPUT }}

# Get a specific field (e.g. "name")
{{ INPUT name }}
```

SEND

Send a message to another agent and get the response.

```
# Standard Form
{{ SEND agent MESSAGE input }}

# Shortcut (implies MESSAGE INPUT)
{{ SEND agent }}

# Block Form (send a custom message body)
{{ SEND agent MESSAGE }}
  Here is a long message...
{{ END }}
```

FACT

Retrieve a value from the agent's memory.

```
# Get a fact by name
{{ FACT name }}
```

IF

Make a decision based on a condition.

```
# Statement Form (inline)
{{ IF INPUT name IS EMPTY THEN "Unknown" ELSE INPUT name }}

# Block Form (multiline)
{{ IF INPUT name IS EMPTY THEN }}
  I don't know who you are.
{{ ELSE }}
  Hello, {{ INPUT name }}.
{{ END }}
```

LIST

Format a list of items (Facts or Inputs).

```
# Standard Form (Bullets are default)
{{ LIST shopping_list AS BULLETS }}

# Shortcut
{{ LIST shopping_list }}

# Other Formats
{{ LIST shopping_list AS SENTENCES }}
```

LET / USE

Capture a value into a variable (LET) and use it later (USE).

```
# Statement Form
{{ LET time BE SEND time_keeper }}

# Block Form (capture text)
{{ LET greeting BE }}
  Hello {{ INPUT name }}
{{ END }}

# Using the variable
{{ USE time }}
```

Appendix B: Built-in Library Agents

Agencia comes with a set of built-in agents that you can call to do common tasks. These agents are available to your templates automatically.

time

now Returns the current date and time.

- **Syntax**: `{{ SEND now IN time }}`
- **Inputs**:
  - `format` (optional): How to format the time (e.g. "Kitchen", "RFC3339").
  - `timezone` (optional): Specific timezone (e.g. "America/New_York").

date Returns the current date only (e.g. "2024-01-01").

- **Syntax**: `{{ SEND date IN time }}`

clock Returns the current time only (e.g. "12:00:00").

- **Syntax**: `{{ SEND clock IN time }}`

math

random Returns a random number between 0 and 1.

- **Syntax**: `{{ SEND random IN math }}`

coin_flip Returns "Heads" or "Tails".

- **Syntax**: `{{ SEND coin_flip IN math }}`

text

uuid Generates a unique identifier (UUID).

- **Syntax**: `{{ SEND uuid IN text }}`

web

search Perform a web search and return a summary of results.

- **Syntax**: `{{ SEND search IN web }}`
- **Inputs**:
  - `query` (required): The text to search for.

Calling Library Agents

Library agents are special agents that are always available. Use the `IN` keyword to specify the library where the agent can be found.

Syntax: `{{ SEND <agent> IN <library> MESSAGE <value> }}`

Example:

```
template: |
  I checked the clock and the time is {{ SEND now IN time }}.
  I also flipped a coin and it came up {{ SEND coin_flip IN math }}.
```